ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ
----------o0o----------

LUẬN VĂN TỐT NGHIỆP ĐẠI HỌC

THIẾT KẾ VÀ TÍCH HỢP DMA CONTROLLER VÀ BỘ TĂNG TỐC PHẦN CỨNG CHACHA20-POLY1305 CHO RISC-V

GVHD: PGS.TS Trần Hoàng Linh
SVTH: Lê Tuấn Khanh
MSSV: 2211485

TP. HỒ CHÍ MINH, THÁNG NĂM 20

ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

----- ☆ -----

Số: _____ /BKĐT
Khoa: **Điện – Điện tử**
Bộ Môn: **Điện Tử**

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc.

----- ☆ -----

NHIỆM VỤ LUẬN VĂN TỐT NGHIỆP

1. HỌ VÀ TÊN: LÊ TUẤN KHANH MSSV: 2211485

2. NGÀNH: **ĐIỆN TỬ - VIỄN THÔNG** LỚP : DD22DVT2
3. Đề tài: Thiết kế và tích hợp DMA Controller và bộ tăng tốc phần cứng ChaCha20-Poly1305 cho RISC-V
4. Nhiệm vụ:

   .....
   .....
   .....
   .....
   .....
   .....
5. Ngày giao nhiệm vụ luận văn: .....
6. Ngày hoàn thành nhiệm vụ: .....
7. Họ và tên người hướng dẫn: Phần hướng dẫn

   ....
   ....

Nội dung và yêu cầu LVTN đã được thông qua Bộ Môn.

Tp.HCM, ngày..... tháng..... năm 20

CHỦ NHIỆM BỘ MÔN **NGƯỜI HƯỚNG DẪN CHÍNH**

PHẦN DÀNH CHO KHOA, BỘ MÔN:
Người duyệt (chấm sơ bộ):.....
Đơn vị:.....
Ngày bảo vệ : .....
Điểm tổng kết: .....
Nơi lưu trữ luận văn: .....

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc đến Thầy Trần Hoàng Linh , giảng viên Khoa Điện tử, Trường Đại học Bách Khoa – Đại học Quốc gia Thành phố Hồ Chí Minh. Sự hướng dẫn tận tình, những nhận xét chuyên môn sâu sắc và sự hỗ trợ không ngừng của các Thầy trong suốt quá trình thực hiện đồ án tốt nghiệp đã đóng vai trò then chốt giúp chúng em hoàn thành đề tài một cách thành công.

Chúng em cũng xin gửi lời cảm ơn chân thành đến Quý Thầy/Cô trong Khoa Điện tử, những người đã truyền đạt kiến thức và kinh nghiệm quý báu, tạo nền tảng lý thuyết vững chắc để chúng em có thể thực hiện tốt đồ án này.

Mặc dù chúng em đã cố gắng hết sức để hoàn thiện đồ án một cách đầy đủ và có hệ thống, tuy nhiên không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Quý Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và chuẩn bị tốt hơn cho con đường nghề nghiệp trong tương lai.

Tp. Hồ Chí Minh, ngày tháng năm .

Sinh viên

TÓM TẮT LUẬN VĂN

Luận văn này trình bày quá trình thiết kế và tích hợp hai thành phần phần cứng mới - Bộ điều khiển DMA (Direct Memory Access) hai kênh và Bộ tăng tốc phần cứng ChaCha20-Poly1305.

Nền tảng hệ thống được kế thừa bao gồm lõi vi xử lý RISC-V RV32I với pipeline 5 tầng (IF → ID → EX → MEM → WB) và hệ thống bus TileLink Uncached Lightweight (TL-UL) với cấu hình crossbar 2 master, 6 slave. Toàn bộ thiết kế được mô tả bằng ngôn ngữ SystemVerilog và mô phỏng trên công cụ ModelSim/Questa.

DMA Controller được thiết kế với kiến trúc 2 kênh độc lập, mỗi kênh điều khiển bởi một máy trạng thái, đóng vai trò kép: là Slave để CPU cấu hình thông qua các thanh ghi MMIO, và là Master để chủ động truy cập bộ nhớ mà không cần can thiệp của CPU.

ChaCha20 Hardware Accelerator được hiện thực theo chuẩn RFC 8439, sử dụng 4 module Quarter Round (QR) chạy song song trong mỗi chu kỳ, hoàn thành một block 512-bit chỉ trong 23 chu kỳ xung nhịp.

Poly1305 MAC Hardware Accelerator được thiết kế với bộ nhân tuần tự 4 chu kỳ và mạch rút gọn Barrett hai bước modulo, hoàn thành xác thực một block 16 byte trong 6 chu kỳ xung nhịp.

Toàn bộ hệ thống được tích hợp và kiểm tra thành công ở ba cấp độ: unit test từng module, integration test qua crossbar thực tế, và system test với chương trình chạy trực tiếp trên lõi RISC-V.

MỤC LỤC

1. GIỚI THIỆU 1
1.1 Tổng quan.....1
1.2 Tình hình nghiên cứu trong và ngoài nước .....**Error! Bookmark not defined.**
1.3 Nhiệm vụ luận văn 2
2. LÝ THUYẾT 3
3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG 9
4. THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM (NẾU CÓ).....**Error! Bookmark not defined.**
5. KẾT QUẢ THỰC HIỆN.....**Error! Bookmark not defined.**
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 29
6.1 Kết luận 29
6.2 Hướng phát triển 29
7. TÀI LIỆU THAM KHẢO..... 29
8. PHỤ LỤC..... 29

DANH SÁCH HÌNH MINH HỌA

Hình 5-1 Kết quả thi công .....**Error! Bookmark not defined.**

Hình 5-2 Kết quả mô phỏng.....**Error! Bookmark not defined.**

DANH SÁCH BẢNG SỐ LIỆU

Bảng 1 Thông số hệ thống ..... **Error! Bookmark not defined.**

1. GIỚI THIỆU

1.1 Tổng quan

Trong kỷ nguyên IoT, bảo mật dữ liệu không còn là một tính năng tùy chọn mà đã trở thành yêu cầu bắt buộc đối với mọi thiết bị nhúng. Các nút mạng IoT thường thu thập và truyền tải dữ liệu nhạy cảm qua các kênh giao tiếp không an toàn, khiến chúng dễ dàng trở thành mục tiêu của các cuộc tấn công đánh cắp hoặc giả mạo thông tin. Để giải quyết bài toán này, các tiêu chuẩn bảo mật hiện đại như TLS 1.3 và giao thức VPN WireGuard đã tiêu chuẩn hóa và ưu tiên sử dụng thuật toán AEAD (Authenticated Encryption with Associated Data) ChaCha20-Poly1305 thay thế cho các chuẩn AES truyền thống trên các thiết bị giới hạn tài nguyên.

Việc triển khai mật mã hoàn toàn bằng phần mềm (Software Cryptography) trên các bộ vi xử lý nhúng (như RISC-V) bộc lộ nhiều điểm yếu chí mạng. Các vi xử lý phải tiêu tốn hàng ngàn chu kỳ lệnh (instructions) cho các phép toán dịch bit, cộng module và logic XOR phức tạp, dẫn đến tiêu thụ năng lượng lớn, độ trễ xử lý cao và làm giảm hiệu năng hệ thống một cách trầm trọng. Thêm vào đó, việc luân chuyển một lượng lớn dữ liệu giữa bộ nhớ và ngoại vi thông qua CPU tạo ra "nút thắt cổ chai" (I/O Bottleneck) nghiêm trọng.

Do đó, xu hướng tất yếu trong thiết kế vi mạch hiện nay là tích hợp các bộ tăng tốc mật mã phần cứng (Hardware Crypto Accelerators) trực tiếp vào Hệ thống trên chip (SoC). Bằng cách thiết kế các lõi IP ChaCha20 và Poly1305 chuyên biệt kết hợp cùng Bộ điều khiển truy cập bộ nhớ trực tiếp (DMA Controller) và giao tiếp qua hệ thống Bus tiêu chuẩn công nghiệp, SoC có thể vừa đảm bảo tính bảo mật dữ liệu tuyệt đối (chống đọc trộm và chống giả mạo), vừa giải phóng hoàn toàn vi xử lý khỏi gánh nặng tính toán và luân chuyển dữ liệu.

1.2 Tình hình nghiên cứu trong và ngoài nước

Trong những năm gần đây, kiến trúc tập lệnh mở RISC-V đã thu hút sự quan tâm rộng rãi trong lĩnh vực thiết kế vi mạch nhờ tính linh hoạt và khả năng mở rộng. Nhiều nền tảng mã nguồn mở như Rocket Chip (UC Berkeley) và CVA6 (OpenHW Group), cùng với các sản phẩm thương mại từ SiFive, đã chứng minh tính khả thi của việc xây dựng hệ thống xử lý dựa trên RISC-V, từ mức nghiên cứu đến ứng dụng thực tế.

Song song với đó, các nghiên cứu về tăng tốc phần cứng cho thuật toán mật mã cũng được đẩy mạnh, đặc biệt đối với các thuật toán như ChaCha20 và Poly1305. Một số công trình đã đề xuất các kiến trúc phần cứng chuyên dụng nhằm cải thiện thông lượng và tối ưu tài nguyên phần cứng khi triển khai trên FPGA và ASIC, cho thấy tiềm năng thay thế hoặc bổ sung cho các giải pháp truyền thống như AES-GCM trong một số ứng dụng cụ thể.

Tại Việt Nam, đặc biệt tại Trường Đại học Bách Khoa TP.HCM, việc nghiên cứu và thiết kế hệ thống dựa trên RISC-V đã được triển khai trong các đồ án học phần và đồ án tốt nghiệp. Các công trình trước đây chủ yếu tập trung vào việc xây dựng lõi vi xử lý từ đơn chu kỳ đến pipeline, cũng như tích hợp các ngoại vi cơ bản như UART, SPI và I2C.

Tuy nhiên, phần lớn các hệ thống này vẫn sử dụng phương pháp truyền dữ liệu dựa trên polling hoặc interrupt, dẫn đến hiệu năng chưa tối ưu khi xử lý các tác vụ có khối lượng dữ liệu lớn. Bên cạnh đó, việc tích hợp các thành phần trong hệ thống thường chưa tuân theo các chuẩn giao tiếp bus hiện đại, làm hạn chế khả năng mở rộng và tái sử dụng thiết kế.

Do đó, việc nghiên cứu xây dựng một hệ thống SoC tích hợp bộ điều khiển DMA đa kênh kết hợp với các khối tăng tốc mật mã, sử dụng chuẩn giao tiếp bus hiện đại như TileLink-UL, là một hướng tiếp cận cần thiết. Hướng đi này không chỉ nhằm cải thiện hiệu năng hệ thống thông qua xử lý song song mà còn góp phần chuẩn hóa kiến trúc và nâng cao khả năng mở rộng trong các thiết kế SoC dựa trên RISC-V.

1.3 Nhiệm vụ luận văn

Kế thừa các kết quả nghiên cứu từ các học kỳ trước - cụ thể là đã thiết kế hoàn thiện lõi vi xử lý RISC-V (Pipelined) và hệ thống mạng liên kết chuẩn công nghiệp (TileLink-UL Crossbar) — đồ án tốt nghiệp này thu hẹp phạm vi vào việc nghiên cứu kiến trúc bảo mật, thiết kế các IP lõi mới và tích hợp toàn diện hệ thống. Các nhiệm vụ trọng tâm được xác định bao gồm:

Nhiệm vụ 1: Thiết kế Bộ điều khiển truy cập bộ nhớ trực tiếp (DMA Controller) đa kênh. Phát triển lõi IP DMA đóng vai trò linh hoạt: nhận lệnh cấu hình từ vi xử lý (Slave) và tự động chiếm quyền điều khiển Bus để luân chuyển dữ liệu (Master). Thiết kế kiến trúc 2 kênh hoạt động song song nhằm giải quyết triệt để nút thắt cổ chai I/O.

Nhiệm vụ 2: Thiết kế lõi tăng tốc phần cứng ChaCha20. Phân tích thuật toán và xây dựng vi kiến trúc (Datapath, FSM) thực thi thuật toán mã hóa dòng (Stream Cipher) ChaCha20 tốc độ cao, tuân thủ nghiêm ngặt đặc tả của IETF RFC 8439.

Nhiệm vụ 3: Thiết kế lõi tăng tốc phần cứng Poly1305 MAC. Xây dựng bộ tạo Mã xác thực thông điệp (Message Authentication Code) Poly1305 theo RFC 8439, kết nối luồng dữ liệu mật mã với khối ChaCha20 để tạo thành nền tảng mã hóa xác thực toàn vẹn (AEAD).

Nhiệm vụ 4: Tích hợp Hệ thống trên chip (SoC Integration). Đấu nối hoàn chỉnh các IP phần cứng mới (DMA, ChaCha20, Poly1305) cùng các ngoại vi cơ bản (SRAM, UART) vào hệ thống CPU và Bus TileLink sẵn có. Thiết lập bản đồ bộ nhớ (Memory Map) phân tán và cơ chế ngắt trực tiếp để tạo thành một kiến trúc SoC tối ưu, liền mạch.

Nhiệm vụ 5: Kiểm thử mô phỏng và Đánh giá hiệu năng (Verification & Benchmarking). Thiết lập môi trường Testbench cấp độ hệ thống để chứng minh tính đúng đắn của phần cứng mật mã so với mô hình tham chiếu (Reference Model). Định lượng các chỉ số hiệu năng (tổng chu kỳ xung nhịp, số lệnh CPU tiết kiệm được (IPC)) thông qua phương pháp A/B Testing giữa hệ thống dùng CPU định tuyến và hệ thống có sự can thiệp của DMA.

2. LÝ THUYẾT

2.1 Giao thức truyền dữ liệu TileLink Uncached Lightweight

2.1.1 Tổng quan

TileLink là một chuẩn liên kết on-chip (chip-scale interconnect) mã nguồn mở và miễn phí, được thiết kế bởi SiFive và cộng đồng RISC-V nhằm cung cấp cơ chế truy cập bộ nhớ ánh xạ (memory-mapped) giữa nhiều Master và Slave. Giao thức này được xây dựng trên mô hình kết nối điểm-điểm (point-to-point) linh hoạt, định nghĩa các giao dịch thông qua các kênh truyền độc lập với cơ chế bắt tay (handshake) Ready/Valid.

Trong họ giao thức này, TileLink Uncached Lightweight (TL-UL) là mức tuân thủ đơn giản nhất, được thiết kế đặc biệt phục vụ các truy cập bộ nhớ không cache (uncached memory access). Chuẩn TL-UL loại bỏ các kênh phức tạp phục vụ quản lý cache, giúp giảm thiểu chi phí phần cứng và rất phù hợp để kết nối các thanh ghi điều khiển ngoại vi (MMIO), bộ nhớ đơn giản, hoặc các thiết bị Slave có độ phức tạp thấp như UART, SPI.

2.1.2 Cấu trúc kênh giao tiếp trong TL-UL

Giao thức TL-UL được rút gọn tối đa, chỉ sử dụng 2 kênh (Channel A và Channel D) trong tổng số 5 kênh tiêu chuẩn của TileLink.

Kênh A (Request Channel): Đảm nhiệm việc truyền tải các yêu cầu từ Master tới Slave. Kênh này cho phép Master phát hành nhiều yêu cầu liên tiếp (outstanding transactions) mà không cần chờ phản hồi trước đó, giúp hệ thống có khả năng pipeline tốt và giảm đường găng (critical path). Các tín hiệu cốt lõi trên kênh A bao gồm a_opcode (mã lệnh 3-bit), a_address (địa chỉ byte đích), a_data (dữ liệu payload tải đi), a_mask (mặt nạ chọn làn byte hợp lệ), cùng cặp tín hiệu điều khiển luồng a_valid và a_ready.

Kênh D (Response Channel): Chịu trách nhiệm mang các thông điệp phản hồi từ Slave về Master. Giao thức quy định nghiêm ngặt rằng mỗi yêu cầu trên kênh A luôn phải có đúng một phản hồi tương ứng trên kênh D. Kênh D chứa các tín hiệu quan trọng như d_opcode (mã lệnh phản hồi), d_data (dữ liệu tải về), d_denied (cờ báo lỗi Slave từ chối phục vụ), cùng cặp tín hiệu bắt tay d_valid và d_ready.

2.1.3 Các tập lệnh (Opcodes) cơ bản

Chuẩn TL-UL chỉ hỗ trợ nhóm lệnh ACCESS cơ bản phục vụ thao tác đọc và ghi. Quá trình này được thực hiện thông qua sự đối đáp giữa các opcode trên hai kênh:

- Get (Đọc dữ liệu): Master gửi yêu cầu Get trên kênh A để đọc dữ liệu tại một địa chỉ xác định. Slave bắt buộc phải trả về dữ liệu trực tiếp thông qua thông điệp AccessAckData trên kênh D; giao thức không cho phép trường hợp lệnh đọc trả về phản hồi hợp lệ mà không kèm theo dữ liệu.

- PutFullData (Ghi toàn bộ): Master yêu cầu ghi toàn bộ độ rộng bus tại địa chỉ đích mà không sử dụng mặt nạ byte, cho phép Slave ghi đè dữ liệu trực tiếp. Thao tác này có logic đơn giản, tối ưu cho việc ghi nguyên một word. Sau khi ghi thành công, Slave phản hồi bằng thông điệp AccessAck trên kênh D.

- PutPartialData (Ghi một phần): Master yêu cầu ghi dữ liệu theo từng byte lẻ thông qua tín hiệu a_mask để chỉ định byte nào hợp lệ. Slave phải đọc giá trị hiện tại, trộn (merge) với dữ liệu mới theo mặt nạ, rồi mới tiến hành ghi lại. Đây là tập lệnh bắt buộc để thao tác chính xác với các thanh ghi MMIO khi phần mềm cần

thay đổi các bit hoặc byte riêng lẻ. Tương tự như PutFullData, kết quả được báo cáo về Master bằng gói AccessAck.

2.2 Truy cập bộ nhớ trực tiếp DMA – Direct Access Memory

2.2.1 Khái niệm và Vai trò

Truy cập bộ nhớ trực tiếp (DMA) là một cơ chế phần cứng cho phép các thiết bị ngoại vi hoặc bộ tăng tốc (accelerators) truyền tải dữ liệu trực tiếp đến hoặc đi từ bộ nhớ chính (SRAM/DRAM) mà không cần sự can thiệp liên tục của vi xử lý trung tâm (CPU).

Trong các hệ thống nhúng thông thường, việc luân chuyển dữ liệu thường được CPU thực hiện thông qua hai phương pháp: thăm dò (Polling) hoặc ngắt (Interrupt). Cả hai phương pháp này đều bộc lộ nhược điểm lớn khi xử lý khối lượng dữ liệu tốc độ cao (như mã hóa thông điệp). Polling khiến CPU bị treo (stall) hoàn toàn trong các vòng lặp chờ đợi, làm lãng phí hàng ngàn chu kỳ xung nhịp và gây sụt giảm nghiêm trọng chỉ số IPC (Instructions Per Cycle). Ngắt, mặc dù giải phóng CPU tạm thời, nhưng lại sinh ra chi phí trễ chuyển đổi ngữ cảnh (context-switching overhead).

Bằng cách sử dụng Bộ điều khiển DMA, hệ thống giải quyết triệt để "nút thắt cổ chai" I/O. CPU chỉ đóng vai trò cấu hình ban đầu (địa chỉ nguồn, đích, số lượng byte) và phát lệnh khởi động. Sau đó, DMA sẽ đóng vai trò là một Bus Master độc lập, chủ động chiếm quyền điều khiển hệ thống liên kết mạng (như TileLink) để bơm dữ liệu. Trong thời gian đó, CPU hoàn toàn rảnh rỗi để thực thi các tác vụ tính toán khác hoặc chuyển sang trạng thái tiết kiệm năng lượng.

2.2.2 Memory-Mapped I/O (MMIO) và Cơ chế định tuyến địa chỉ

Trong kiến trúc SoC, Memory-Mapped I/O (MMIO) là phương pháp ánh xạ các thanh ghi điều khiển và bộ đệm dữ liệu của các thiết bị ngoại vi vào chung một không gian địa chỉ vật lý với bộ nhớ RAM. Nhờ MMIO, DMA có thể giao tiếp với các bộ tăng tốc phần cứng thông qua các tập lệnh đọc/ghi tiêu chuẩn mà không cần các đường tín hiệu điều khiển chuyên biệt.

Quá trình luân chuyển dữ liệu của DMA phụ thuộc rất lớn vào cơ chế tịnh tiến địa chỉ (Address Increment) đối với từng loại thiết bị:

- Giao tiếp với RAM (Copy RAM): Bộ nhớ RAM là một mảng lưu trữ tuyến tính. Để sao chép một mảng dữ liệu, địa chỉ nguồn hoặc đích của DMA phải liên tục

được cộng thêm độ rộng của một word dữ liệu (ví dụ +4 bytes) sau mỗi giao dịch thành công. Nếu không tịnh tiến, dữ liệu mới sẽ ghi đè lên dữ liệu cũ tại cùng một ô nhớ.

- Giao tiếp với FIFO / Bộ tăng tốc: Khác với RAM, các ngoại vi như UART TX/RX FIFO hoặc cổng nạp dữ liệu của lõi mã hóa thường chỉ bộc lộ một địa chỉ MMIO duy nhất. Khi DMA ghi dữ liệu vào địa chỉ này, con trỏ nội bộ (internal pointer) của FIFO sẽ tự động tịnh tiến phần cứng bên trong. Do đó, kênh DMA phải được cấu hình giữ nguyên cấu trúc địa chỉ tĩnh (Static Address) trong suốt chu kỳ burst. Việc ép cộng địa chỉ trong trường hợp này sẽ dẫn đến lỗi truy cập vùng nhớ không hợp lệ (Bus Error).

2.2.3 Cơ chế trọng tài (Arbiter) trong DMA đa kênh

Khi hệ thống có nhiều thiết bị hoặc nhiều kênh DMA (ví dụ kênh CH0 nạp Plaintext, kênh CH1 thu thập Ciphertext) cùng muốn truy cập vào Bus, sự xung đột là không thể tránh khỏi. Bộ phân xử trọng tài (Arbiter) được tích hợp để giải quyết bài toán cấp phát quyền Master.

Một trong những thuật toán phổ biến và công bằng nhất được áp dụng trong phần cứng là Round-Robin. Trọng tài Round-Robin duy trì một cờ trạng thái lưu trữ kênh vừa được phục vụ gần nhất (last_served). Khi có nhiều kênh cùng phát tín hiệu yêu cầu (req), Arbiter sẽ duyệt ưu tiên theo vòng tròn, đảm bảo không có bất kỳ kênh nào bị chiếm dụng Bus vĩnh viễn (starvation). Khi một kênh hoàn tất chu kỳ đọc-ghi của mình, quyền làm chủ Bus sẽ được chuyển giao mượt mà sang kênh tiếp theo trong hàng đợi.

2.3 Thuật toán mã hóa dòng ChaCha20

2.3.1 Tổng quan và Ứng dụng

ChaCha20 là một thuật toán mã hóa dòng (Stream Cipher) được phát triển bởi Daniel J. Bernstein, hiện đã được tiêu chuẩn hóa toàn cầu thông qua IETF RFC 8439. Thuật toán này ra đời nhằm khắc phục các điểm yếu về hiệu năng phần mềm và khả năng chống lại các cuộc tấn công Timing-attack của dòng mã hóa RC4 và AES.

ChaCha20 cung cấp mức độ bảo mật 256-bit và được thiết kế tối ưu hóa cho vi kiến trúc của vi xử lý, không sử dụng các bảng tra cứu (Look-up Table) vốn dễ gây rò rỉ thông tin qua bộ đệm (Cache). Ngày nay, ChaCha20 được ứng dụng rộng rãi làm tiêu chuẩn bảo mật

mặc định trong các giao thức mạng tốc độ cao như TLS 1.3, QUIC, và mạng riêng ảo WireGuard.

2.3.2 Cấu trúc trạng thái

Lõi của ChaCha20 hoạt động dựa trên một ma trận trạng thái nội bộ có kích thước 4x4. Ma trận này chứa 16 phần tử, mỗi phần tử là một từ dữ liệu 32-bit (tổng cộng 512 bit), được khởi tạo với cấu trúc nghiêm ngặt sau:

- 4 Word đầu tiên (0-3): Các hằng số ma thuật ("expand 32-byte k") nhằm ngăn chặn các cấu trúc khóa yếu.
- 8 Word tiếp theo (4-11): Khóa bí mật 256-bit (Key).
- 1 Word (12): Bộ đếm khối 32-bit (Block Counter), tự động tăng lên sau mỗi khối 64-byte được xử lý để đảm bảo dòng khóa sinh ra không bao giờ lặp lại.
- 3 Word cuối (13-15): Giá trị sử dụng một lần (Nonce) 96-bit, đảm bảo tính duy nhất cho mỗi phiên mã hóa.

2.3.3 Hàm Quarter Round (QR)

Động cơ tính toán cốt lõi của ChaCha20 là hàm Quarter Round (QR). Hàm này biến đổi 4 từ dữ liệu đầu vào (a, b, c, d) thông qua một chuỗi các phép toán ARX (Add - Rotate - XOR) được tối ưu hóa cho phần cứng:

$$a = a + b\,; d = (d \oplus a) \lll 16$$

$$c = c + d\,; b = (b \oplus c) \lll 12$$

$$a = a + b\,; d = (d \oplus a) \lll 8$$

$$c = c + d\,; b = (b \oplus a) \lll 7$$

Trong đó, $\oplus$ là phép XOR bitwise, và $\lll$ là phép xoay vòng trái (Left Rotation). Các phép toán này không bao hàm các lệnh nhân hoặc chia phức tạp, cho phép triển khai mạch số logic cực kỳ nhanh gọn và song song hóa dễ dàng.

2.3.4 Vòng lặp (Rounds)

Thuật toán thực thi tổng cộng 20 vòng lặp (Rounds) để xáo trộn hoàn toàn ma trận trạng thái, tương đương với 10 vòng lặp kép (Double-rounds).

Mỗi Double-round bao gồm:

- Column round (Vòng cột - Số lẻ): Áp dụng hàm QR trên 4 cột dọc của ma trận.
- Diagonal round (Vòng chéo - Số chẵn): Áp dụng hàm QR trên 4 đường chéo của ma trận.

Sau 20 vòng, ma trận kết quả sẽ được cộng modulo 2^{32} với ma trận khởi tạo ban đầu để sinh ra Khối dòng khóa (Keystream Block) 512-bit. Bản rõ (Plaintext) sẽ được XOR với Keystream này để tạo thành bản mã (Ciphertext).

2.4 Mã xác thực thông điệp Poly1305 và chuẩn AEAD

2.4.1 Mã xác thực Poly1305

Mã hóa đơn thuần (như sử dụng độc lập ChaCha20) chỉ đảm bảo tính bảo mật (Confidentiality) mà không cung cấp cơ chế kiểm tra tính toàn vẹn (Integrity). Hacker có thể chặn bắt gói tin và thay đổi ngẫu nhiên các bit của bản mã. Poly1305 là một thuật toán sinh Mã xác thực thông điệp (MAC), đóng vai trò như một "chữ ký số" đính kèm vào dữ liệu để phát hiện ngay lập tức mọi sự can thiệp trái phép.

Poly1305 đánh giá các khối dữ liệu 16-byte thông qua một phương trình toán học trên trường Galois hữu hạn. Phương trình cốt lõi cho mỗi chu kỳ hấp thụ khối dữ liệu là:

$$acc = \big((acc + block) \times r\big) \pmod P$$

Trong đó:

- *acc* là thanh ghi tích lũy (Accumulator), khởi tạo bằng 0.
- *block* là khối dữ liệu đầu vào.
- *r* là phần đầu tiên của khóa bảo mật.
- *P* là số nguyên tố Mersenne $2^{130} - 5$, đóng vai trò định tuyến giới hạn modulo.

2.4.2 Quy tắc Clamping và Khóa bảo mật

Khóa đầu vào của Poly1305 bao gồm cặp khóa (r, s), mỗi khóa dài 16 bytes. Trước khi đưa vào phương trình tính toán, khóa *r* bắt buộc phải đi qua quy tắc giới hạn (Clamping). Quá trình này sẽ xóa (clear) các bit cụ thể trên *r* (ví dụ: các bit 3, 7, 11, 15 của các byte quy định) để đảm bảo *r* rơi vào các giá trị toán học đặc biệt. Việc Clamping này không chỉ củng cố sức mạnh bảo mật chống lại các nhóm khóa yếu mà còn tối ưu hóa đáng kể tốc độ của bộ nhân phần cứng. Khóa (r, s)được giữ nguyên và chỉ được cộng vào giá trị *acc* ở bước cuối cùng sau khi đã xử lý hết toàn bộ các khối thông điệp.

2.4.3 Kết hợp mã hóa xác thực AEAD (ChaCha20-Poly1305)

Tiêu chuẩn AEAD (Mã hóa có xác thực kèm dữ liệu liên kết) kết hợp hoàn hảo sức mạnh của cả hai thuật toán trên theo luồng dữ liệu chuẩn RFC 8439:

1. Tạo OTK (One-Time Key): Thuật toán ChaCha20 khởi chạy với Khóa và Nonce của phiên, Block Counter được thiết lập bắt buộc bằng 0. 32 byte đầu tiên của Keystream sinh ra sẽ được dùng làm khóa (r, s) cấp phát trực tiếp cho Poly1305.

2. Mã hóa Plaintext: ChaCha20 tăng Block Counter lên 1 và bắt đầu quá trình XOR Keystream với dữ liệu bản rõ để tạo ra Ciphertext.

3. Hấp thụ dữ liệu vào Poly1305: Lõi Poly1305 tuần tự băm (hash) các đoạn dữ liệu bao gồm: Dữ liệu liên kết (AAD - không mã hóa nhưng cần xác thực), Bản mã (Ciphertext), các chuỗi Padding (đệm cho chẵn 16-byte), và cuối cùng là chiều dài (Lengths) của AAD và Ciphertext.

4. Sinh Tag: Sau khi cộng khóa *s*, hệ thống sinh ra một Tag 16-byte duy nhất. Tag này được đính kèm vào cuối gói tin truyền đi, hoàn tất quy trình bảo mật khép kín vừa chống nghe lén, vừa chống giả mạo.

3. THIẾT KẾ KIẾN TRÚC HỆ THỐNG TỔNG THỂ

3.1 Tổng quan hệ thống

Hệ thống SoC được xây dựng theo kiến trúc bus tập trung, trong đó tất cả các thành phần giao tiếp với nhau thông qua một crossbar switch sử dụng giao thức TileLink Uncached Lightweight (TL-UL). Toàn bộ hệ thống hoạt động trong một miền xung nhịp duy nhất, giúp

đơn giản hóa thiết kế và loại bỏ các vấn đề liên quan đến đồng bộ hóa giữa các miền xung nhịp khác nhau.

Crossbar được cấu hình với 2 master và 6 slave (2M6S):

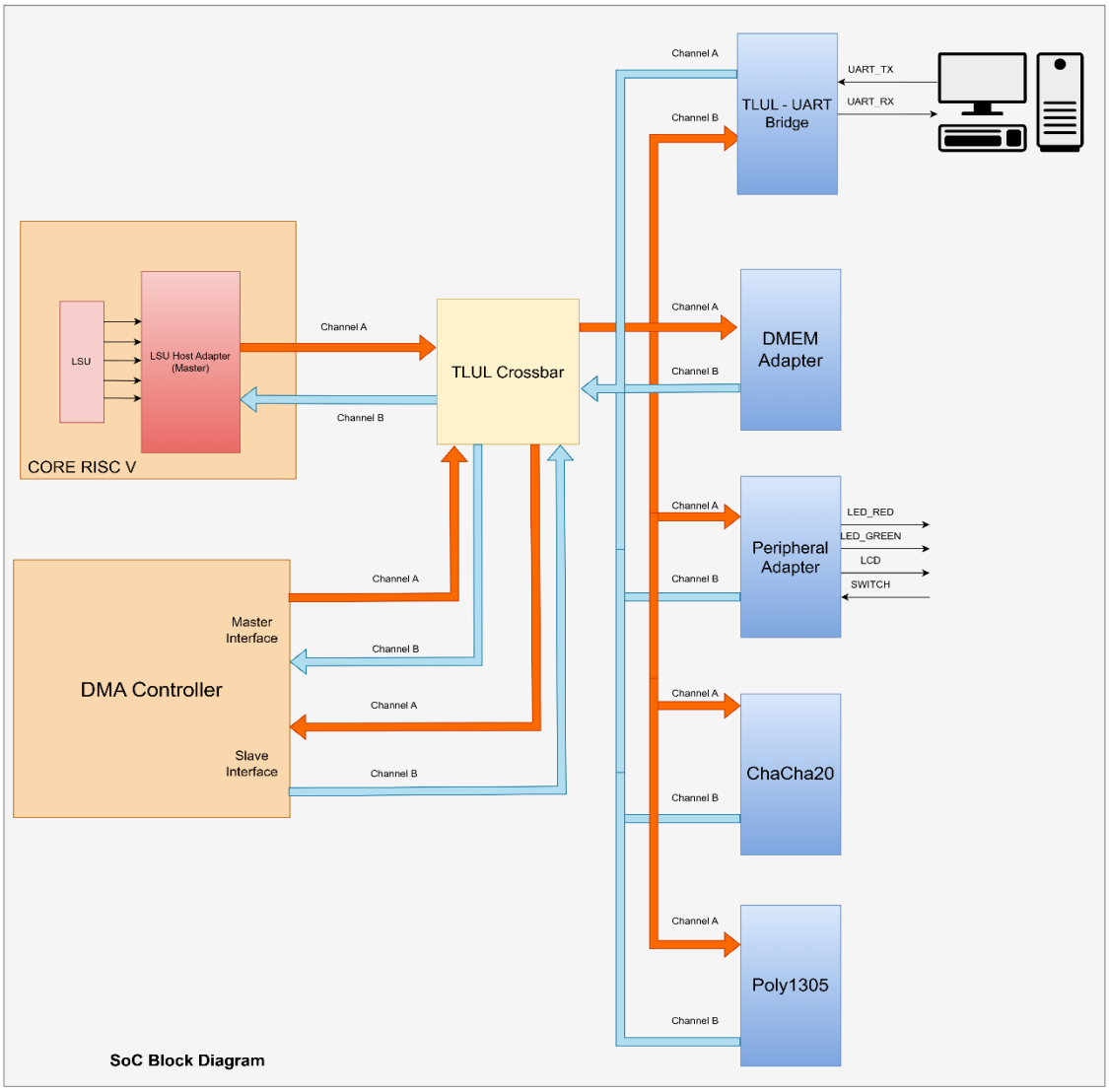
- Master 0 – CPU: Lõi RISC-V pipeline 5 tầng. Phát sinh các giao dịch TileLink tại tầng MEM để thực hiện lệnh load/store.
- Master 1 – DMA Controller: Bộ điều khiển DMA 2 kênh. Chủ động phát sinh các giao dịch đọc/ghi bộ nhớ mà không cần sự can thiệp của CPU.

Sáu slave trong hệ thống được phân bổ địa chỉ như Bảng:

Bảng 1. Memory Mapping cho các Slave trong hệ thống

Slave ID	Tên Module	Địa chỉ base	Kích thước	Mô tả
0	SRAM (Memory)	0x0000_0000	0x0000_7FFF	Bộ nhớ RAM vật lý 32KB (Chứa Instruction & Data).
1	Basic Peripherals	0x1000_0000	0x1000_FFFF	Các ngoại vi cơ bản: LED, HEX, LCD, Switches.
2	UART	0x1002_0000	0x1002_FFFF	Cổng giao tiếp nối tiếp (Có TX FIFO dùng cho DMA).
3	DMA Controller	0x1005_0000	0x1005_FFFF	Bộ điều khiển truy cập bộ nhớ trực tiếp (2 Kênh).
4	ChaCha20	0x1006_0000	0x1006_FFFF	Bộ tăng tốc mã hóa ChaCha20.
5	Poly1305	0x1007_0000	0x1007_FFFF	Bộ tạo mã xác thực thông điệp MAC Poly1305.

Sơ đồ khối tổng thể của hệ thống được trình bày trong Hình 3.1. Mỗi kết nối giữa master/slave và crossbar gồm hai kênh: Channel A mang yêu cầu (request) từ master đến slave, Channel B mang phản hồi (response) theo chiều ngược lại. DMA Controller có vai trò kép: đóng vai trò Master khi chủ động truy cập bộ nhớ qua Master Interface, và đóng vai trò Slave khi CPU ghi vào các thanh ghi cấu hình qua Slave Interface.



Hình 1. Sơ đồ khối tổng quát của hệ thống

Tín hiệu reset bên ngoài rst_n được đồng bộ hóa bằng mạch hai flip-flop trước khi phân phối đến tất cả các module bên trong, đảm bảo tránh hiện tượng metastability khi reset được giải phóng không đồng bộ với xung nhịp hệ thống.Thiết kế, vẽ sơ đồ mạch chi tiết và tính toán từng khối đã nêu trong phần trên.

3.2 Kiến trúc CPU RISC-V

Lõi vi xử lý được xây dựng theo kiến trúc RISC-V RV32I với pipeline 5 tầng: IF → ID → EX → MEM → WB. Phần này trình bày tóm tắt kiến trúc CPU từ công trình kỳ trước, tập trung vào những phần có liên quan trực tiếp đến việc mở rộng hệ thống trong đồ án này.

3.2.1 Cấu trúc pipeline

Pipeline gồm 5 tầng được kết nối qua 4 thanh ghi pipeline: IF/ID, ID/EX, EX/MEM và MEM/WB. Mỗi tầng thực hiện một công việc độc lập trong cùng một chu kỳ xung nhịp, cho phép tối đa 5 lệnh cùng tồn tại trong pipeline ở các giai đoạn khác nhau.

- IF (Instruction Fetch): Đọc lệnh từ bộ nhớ theo địa chỉ PC hiện tại. PC được cập nhật tuần tự (+4) hoặc nhảy đến địa chỉ đích khi có lệnh rẽ nhánh.
- ID (Instruction Decode): Giải mã lệnh, đọc giá trị thanh ghi nguồn từ register file, sinh immediate và các tín hiệu điều khiển.
- EX (Execute): Thực hiện phép tính ALU, tính địa chỉ branch target và phát hiện misprediction.
- MEM (Memory Access): Thực hiện truy cập bộ nhớ thông qua giao thức TileLink-UL. Đây là tầng được mở rộng đáng kể so với thiết kế gốc.
- WB (Write Back): Ghi kết quả trở lại register file.

3.2.2 Xử lý hazard

Pipeline tích hợp hai cơ chế xử lý hazard:

- **Forwarding Unit**: Phát hiện và giải quyết data hazard bằng cách chuyển tiếp kết quả từ tầng MEM hoặc WB trực tiếp về đầu vào tầng EX, tránh phải stall pipeline trong hầu hết các trường hợp. Forwarding unit hỗ trợ cả trường hợp JAL/JALR cần chuyển tiếp giá trị PC+4.
- **Hazard Detection Unit**: Phát hiện các trường hợp không thể giải quyết bằng forwarding, bao gồm load-use hazard (lệnh load theo sau ngay bởi lệnh sử dụng kết quả) và misprediction. Khi phát hiện load-use hazard, đơn vị này chèn một NOP bubble vào tầng ID/EX và giữ nguyên các tầng IF và ID trong một chu kỳ.

3.2.3 Giao tiếp Tilelink tại tầng MEM

Đây là điểm mở rộng quan trọng nhất của CPU trong bối cảnh tích hợp SoC. Thay vì truy cập bộ nhớ trực tiếp, tầng MEM (module MEM_tl) phát sinh các giao dịch TileLink-UL chuẩn:

- Lệnh **load**: Phát gói Get (opcode = 4) trên Channel A với địa chỉ tính toán từ ALU. Pipeline stall cho đến khi nhận được AccessAckData trên Channel D.
- Lệnh **store**: Phát gói PutFullData (opcode = 0) trên Channel A kèm dữ liệu cần ghi. Pipeline stall cho đến khi nhận được AccessAck trên Channel D.

Tín hiệu lsu_stall được kéo lên mức 1 trong suốt thời gian chờ phản hồi TileLink. Tín hiệu này tác động đến toàn bộ pipeline thông qua logic stall/flush:

```
pipeline_stall  = lsu_stall

combined_stall  = stall | pipeline_stall   -- giữ IF, ID, EX

combined_flush  = mispred & ~pipeline_stall -- flush bị trì hoãn khi TL đang bận
```

Thiết kế này đảm bảo rằng nếu phát hiện misprediction trong khi pipeline đang stall do TileLink, lệnh flush sẽ được trì hoãn đến khi giao dịch TileLink hoàn thành, tránh mất dữ liệu hoặc flush không đúng tầng.

3.3 TileLink Crossbar

Module Crossbar đóng vai trò trung tâm kết nối toàn bộ hệ thống, cho phép 2 master giao tiếp với 6 slave thông qua một tập hợp các multiplexer và logic phân xử truy cập bus.

3.3.1 Giải mã địa chỉ

Mỗi yêu cầu từ master mang theo địa chỉ 32-bit. Crossbar kiểm tra các bit địa chỉ cao để xác định slave đích theo quy tắc:

Bảng 2. Bảng giải mã địa chỉ khi đi qua Crossbar

Điều kiện địa chỉ	**Slave được chọn**
addr[31:15] == 0	S0 – DMEM Adapter
addr[27:24] == 4'h1 & addr[23:20] == 4'h0 & addr[19:16] == 4'h0	S1 – Peripheral
addr[23:16] == 8'h02	S2 – UART
addr[23:16] == 8'h05	S3 – DMA Registers
addr[23:16] == 8'h06	S4 – ChaCha20

addr[23:16] == 8'h07	S5 – Poly1305

Kết quả giải mã tạo ra tín hiệu slave_sel[5:0] chỉ định đúng một slave được kích hoạt cho mỗi giao dịch.

3.3.2 Cơ chế phân xử Round-Robin

Khi cả hai master đồng thời phát sinh yêu cầu đến cùng một slave, crossbar sử dụng thuật toán **Round-Robin** để phân xử công bằng. Trạng thái phân xử được quản lý bởi thanh ghi rr_priority: khi bằng 0 thì CPU được ưu tiên, khi bằng 1 thì DMA được ưu tiên. Sau mỗi lần cấp quyền truy cập, rr_priority được đảo chiều để đảm bảo hai master luân phiên nhau khi cùng tranh chấp.

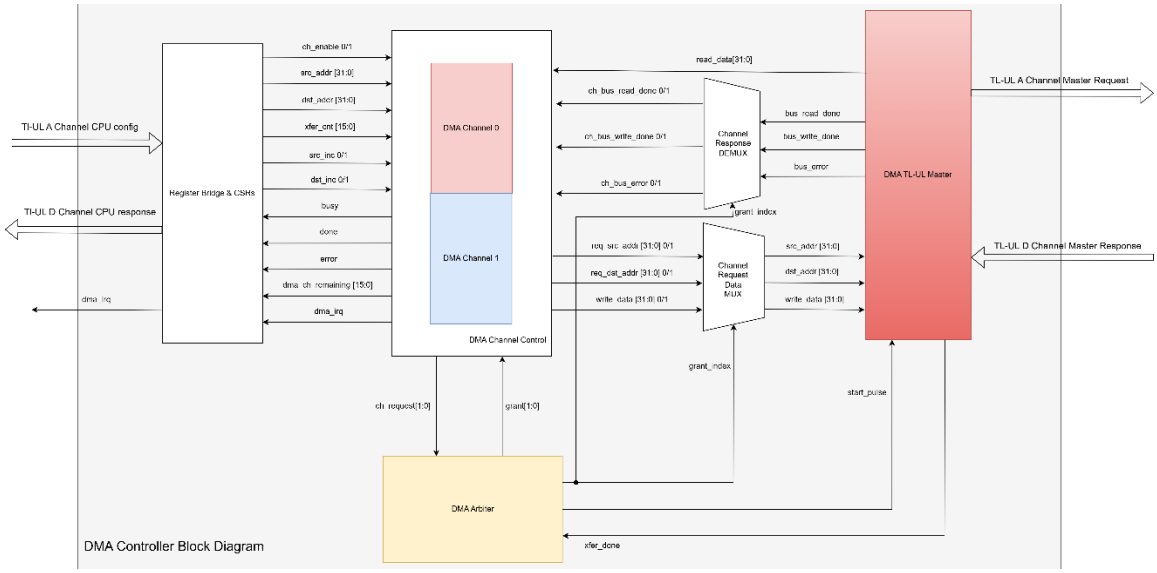
Crossbar hoạt động theo mô hình **transaction-based**: một giao dịch được coi là hoàn thành khi nhận được tín hiệu phản hồi hợp lệ trên Channel D. Trong thời gian một giao dịch đang xử lý, master kia có thể được cấp quyền truy cập các slave khác nếu không xảy ra xung đột.

3.4 Direct Access Memory Controller

Bộ điều khiển DMA (Direct Memory Access) là một thành phần quan trọng trong các hệ thống SoC hiện đại, cho phép truyền dữ liệu trực tiếp giữa bộ nhớ và ngoại vi (hoặc giữa bộ nhớ với bộ nhớ) mà không cần sự can thiệp liên tục của CPU. Thiết kế này sử dụng giao thức TileLink-UL (Uncached Lightweight), một giao thức bus hiện đại, tinh gọn, thường thấy trong các hệ sinh thái RISC-V.

Các thành phần chính bao gồm:

- Register Bridge & CSRs: Chứa các thanh ghi cấu hình (MMIO) và điều khiển logic ngắt (IRQ), nhận tín hiệu từ CPU thông qua bus TL-UL để cấu hình các thanh ghi điều khiển hoạt động của DMA.
- DMA_Controller: Khối thực thi, điều phối giữa các kênh và bus master, bên trong bản chất là các MUX và DEMUX để phân loại tín hiệu từ các Channel.
- DMA_Channel: Máy trạng thái (FSM) quản lý địa chỉ và số lượng từ cần truyền cho mỗi kênh.
- DMA_Arbiter: Trọng tài phân xử quyền truy cập bus theo thuật toán Round-Robin.
- DMA_TLUL_Master: Công cụ chuyển đổi yêu cầu nội bộ thành các transaction chuẩn TileLink-UL, giúp DMA giao tiếp với các Slave khác bên ngoài SoC top.



Hình 2. Sơ đồ khối tổng quản DMA Controller

3.4.1 Giao tiếp Thanh ghi (Register Map & Slave Interface)

CPU cấu hình DMA thông qua các thanh ghi MMIO:

- **CHX_CONTROL:** Bật/tắt kênh, cấu hình tăng địa chỉ (increment).
- **CHX_SRC/DST_ADDR:** Địa chỉ nguồn và đích.
- **CHX_XFER_CNT:** Số lượng từ (32-bit words) cần truyền.
- **IRQ_STATUS:** Quản lý ngắt theo cơ chế W1C (Write 1 to Clear)

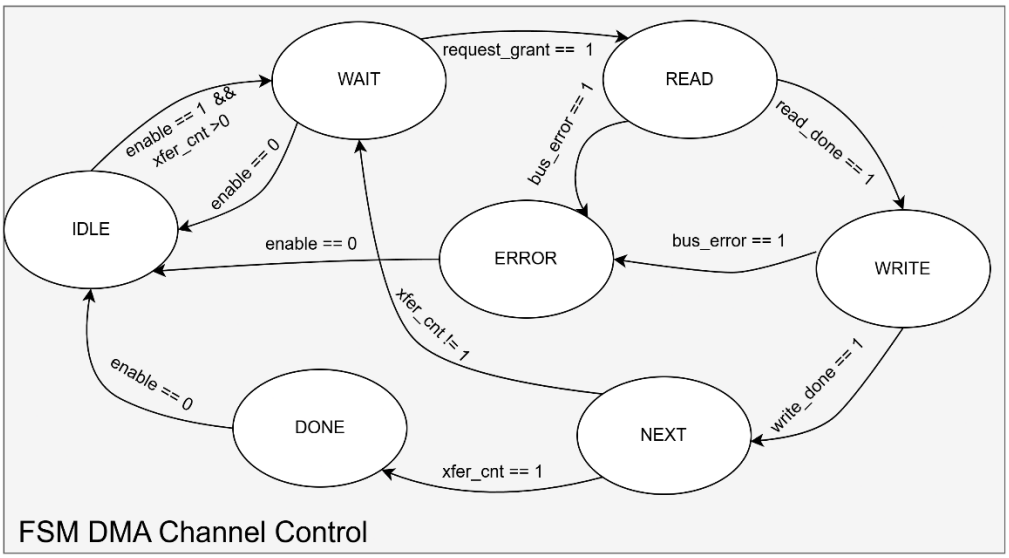
Bảng 3. Địa chỉ Mapping cho các thanh ghi CSRs trong DMA

Offset	Địa chỉ	Channel	Tên thanh ghi	Quyền	Mô tả
+0x00	0x1005_0000	CH0	CH0_CONTROL	R/W	[0]=en, [1]=src_inc, [2]=dst_inc.
+0x04	0x1005_0004	CH0	CH0_STATUS	RO	[0]=busy, [1]=done, [2]=err, [31:16]=remain.
+0x08	0x1005_0008	CH0	CH0_SRC_ADDR	R/W	Địa chỉ nguồn.
+0x0C	0x1005_000C	CH0	CH0_DST_ADDR	R/W	Địa chỉ đích.
+0x10	0x1005_0010	CH0	CH0_XFER_CNT	R/W	Số lượng bytes cần truyền.
+0x20	0x1005_0020	CH1	CH1_CONTROL	R/W	Cấu hình kênh 1
+0x24	0x1005_0024	CH1	CH1_STATUS	RO	Trạng thái kênh 1.
+0x28	0x1005_0028	CH1	CH1_SRC_ADDR	R/W	Địa chỉ nguồn kênh

					1.
+0x2C	0x1005_002C	CH1	CH1_DST_ADDR	R/W	Địa chỉ đích kênh 1.
+0x30	0x1005_0030	CH1	CH1_XFER_CNT	R/W	Số lượng bytes kênh 1.
+0x40	0x1005_0040	IRQ	IRQ_STATUS	W1C	[0]=ch0_irq, [1]=ch1_irq. Ghi 1 để xóa.

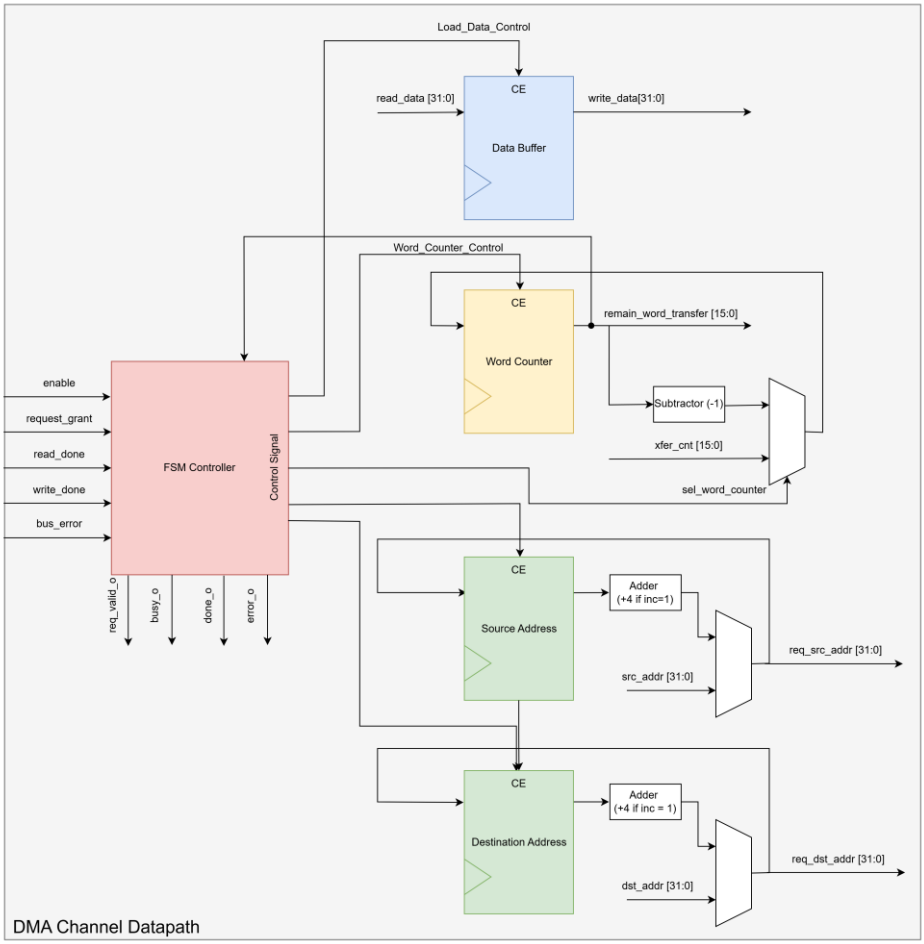
3.4.2 DMA Channel Controller

Mỗi kênh DMA (DMA Channel) hoạt động dựa trên một FSM 6 trạng thái để đảm bảo tính toàn vẹn của dữ liệu:



Hình 3. Máy trạng thái điều khiển hoạt động DMA Controller

- **CH_IDLE:** Chờ lệnh kích hoạt từ CPU.
- **CH_WAIT:** Gửi yêu cầu (request) tới bộ trọng tài.
- **CH_READ:** Thực hiện đọc dữ liệu từ địa chỉ nguồn thông qua Bus Master.
- **CH_WRITE:** Ghi dữ liệu vào địa chỉ đích.
- **CH_NEXT:** Cập nhật địa chỉ, giảm bộ đếm và kiểm tra điều kiện kết thúc.
- **CH_DONE/ERROR:** Kết thúc truyền thành công hoặc báo lỗi bus.



Hình 4. Sơ đồ khối kiến trúc bên trong DMA Channel

CH_IDLE (Trạng thái nghỉ): Kênh nằm chờ phần mềm kích hoạt, dùng logic phát hiện cạnh lên (cfg_enable_i && !enabled_q). Điều này đảm bảo DMA không tự động chạy lại nếu CPU quên xóa cờ enable từ lần truyền trước. Khi thỏa mãn điều kiện, kênh lập tức chốt địa chỉ nguồn (cfg_src_addr_i), đích (cfg_dst_addr_i), số từ cần truyền (cfg_xfer_cnt_i) và xóa cờ lỗi error_q về 0, sau đó chuyển sang CH_WAIT.

CH_WAIT (Chờ phân xử Bus): Phát tín hiệu req_valid_o = 1 để xin quyền truy cập Bus từ khối Arbiter. Kênh sẽ đứng đây cho tới khi Arbiter phản hồi bằng req_grant_i = 1, lập tức sang CH_READ. Đặc biệt, CPU có thể cancel quá trình nếu hạ cfg_enable_i xuống mức 0.

CH_READ (Đọc dữ liệu từ nguồn): Yêu cầu Master đọc từ Bus. Nếu nhận được read_done_i và không có lỗi, dữ liệu từ Bus (read_data_i) sẽ được chốt trực tiếp vào thanh ghi đệm data_buf_q, đồng thời FSM đi thẳng sang CH_WRITE. Nếu bị báo lỗi (bus_error_i), FSM rẽ nhánh về CH_ERROR.

CH_WRITE (Ghi dữ liệu tới đích): Đẩy dữ liệu lưu trong data_buf_q ra Bus tới địa chỉ đích. Hoàn tất ghi (write_done_i = 1), FSM chuyển qua CH_NEXT. Nếu có lỗi Bus sinh ra trong lúc ghi, kênh cũng hủy thao tác và nhảy sang CH_ERROR.

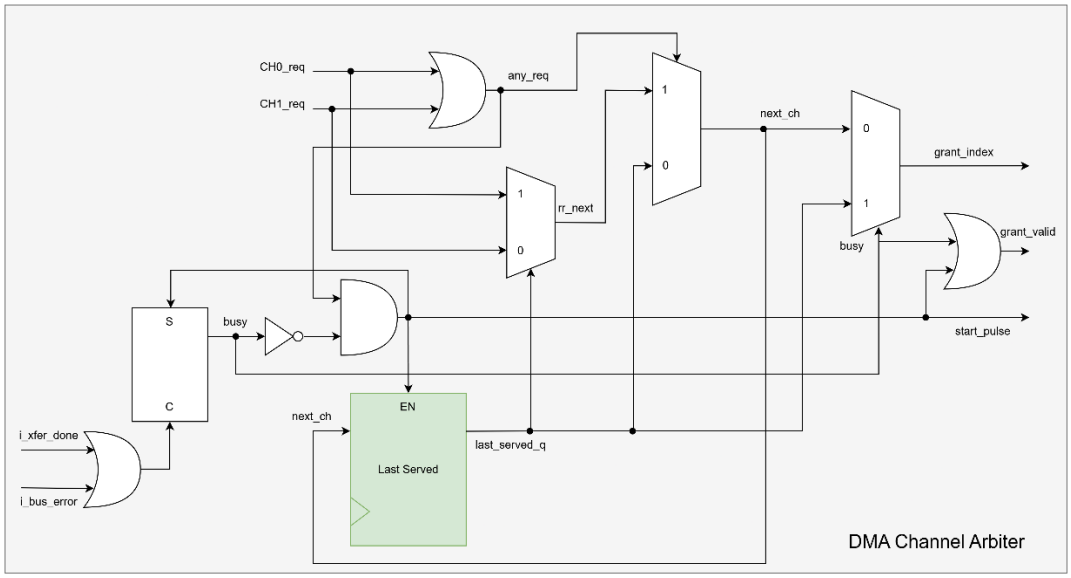
CH_NEXT (Cập nhật địa chỉ & con đếm): Xác định bước đi kế tiếp cho 1 word đã truyền thành công. Địa chỉ nguồn và đích sẽ được cộng thêm 4 (+ 32'd4) nếu CPU có bật bit cfg_src_inc_i / cfg_dst_inc_i. Biến đếm count_q trừ đi 1. Nếu biến đếm ở chu kỳ này đang bằng 1 (nghĩa là sau khi trừ sẽ hết dữ liệu), FSM chuyển về CH_DONE. Nếu vẫn còn dữ liệu (>1), nó lặp lại chu trình bằng cách quay về CH_WAIT để xin Bus.

CH_DONE (Hoàn thành): Kênh đã truyền xong toàn bộ dữ liệu. Xung ngắt hoàn thành (done_o) và xung gọi CPU (irq_o) được tạo ra đúng 1 chu kỳ clock khi FSM vừa bước vào trạng thái này. Khóa cứng kênh tại đây cho đến khi CPU phản hồi bằng cách ghi 0 vào bit cfg_enable_i. Sau đó FSM mới xả về CH_IDLE.

CH_ERROR (Lỗi Bus): Kênh bị lỗi đọc/ghi trên đường truyền. Bật sticky flag error_q = 1 và xuất một xung ngắt lỗi irq_o để gọi CPU xử lý sự cố. Tương tự như CH_DONE, kênh treo ở đây và chờ phần mềm ghi đè tắt cfg_enable_i để reset trạng thái kênh về CH_IDLE.

3.4.3 Bộ Arbiter

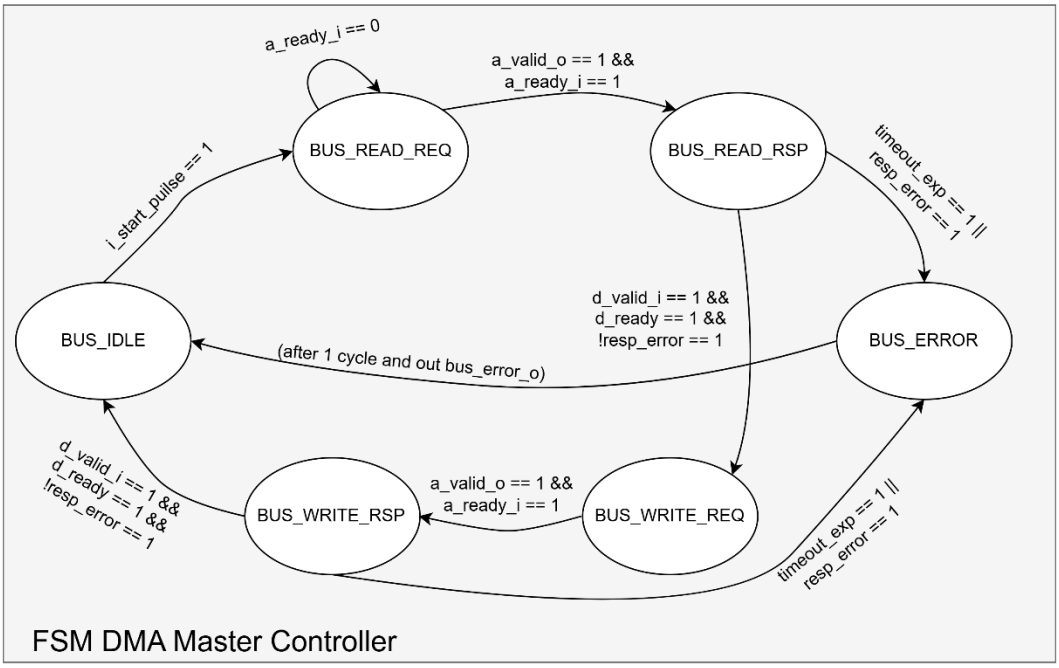
Để đảm bảo tính công bằng khi cả hai kênh cùng yêu cầu truy cập bus, bộ trọng tài sử dụng thuật toán Round-robin. Sau khi một kênh hoàn thành một giao dịch (read + write), quyền ưu tiên sẽ được chuyển sang kênh kế tiếp.



Hình 5. Sơ đồ khối tổng quát bộ Arbiter trong DMA

3.4.4 DMA Master

DMA Master chịu trách nhiệm tạo các gói tin Get (để đọc) và PutFullData (để ghi), giúp giao tiếp với các Slave khác thông qua Bus TL-UL mà không cần đi qua CPU. Nếu Slave không phản hồi trong vòng 1024 chu kỳ, Master sẽ tự động hủy giao dịch và báo lỗi để tránh treo hệ thống. Ngoài ra, Master giám sát các tín hiệu d_denied và d_corrupt từ bus để kích hoạt trạng thái BUS_ERROR.



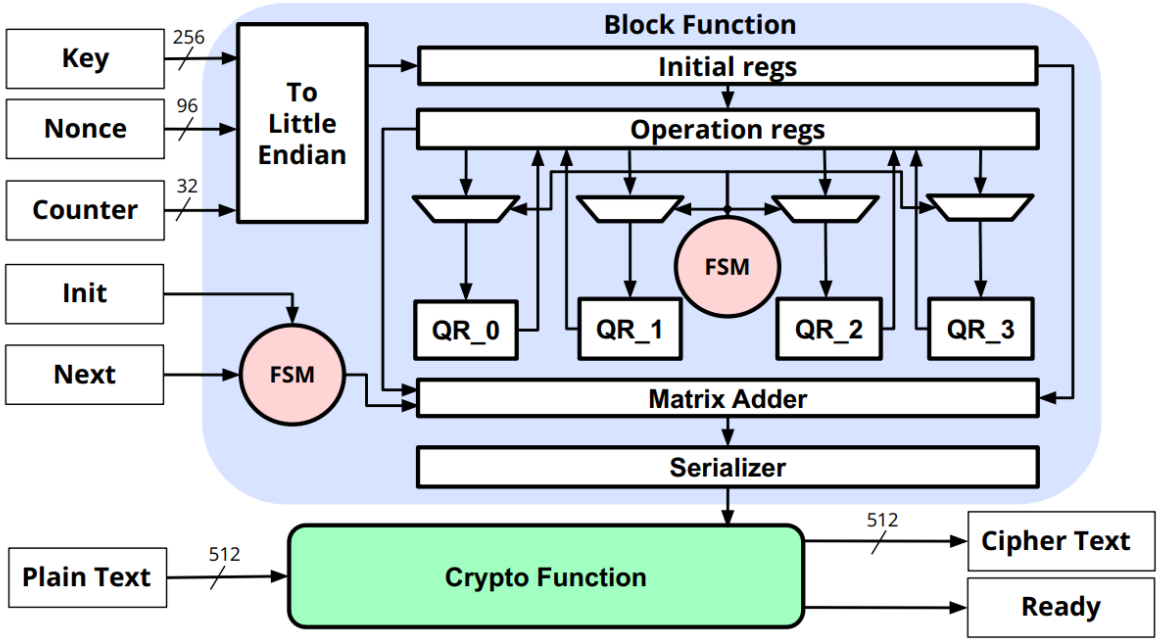
Hình 6. Máy trạng thái biểu thị hoạt động DMA khi là Master

3.5 ChaCha20 Accelerator

3.5.1 Tổng quan

ChaCha20 là thuật toán mã hóa dòng (stream cipher) được định nghĩa trong RFC 8439. Bộ tăng tốc phần cứng ChaCha20 trong hệ thống SoC này bao gồm ba lớp:

- **chacha20_qr**: Module tính toán hàm Quarter Round, hoàn toàn tổ hợp (combinatorial).
- **chacha20_core**: Lõi tính toán điều khiển bởi FSM, sử dụng 4 instance của chacha20_qr chạy song song.
- **tlul_chacha20**: Lớp giao tiếp TileLink-UL, cung cấp register map cho CPU và kết nối với chacha20_core.



Hình 7. Sơ đồ khối tổng quát bộ mã hóa ChaCha20

3.5.2 Hàm Quarter Round

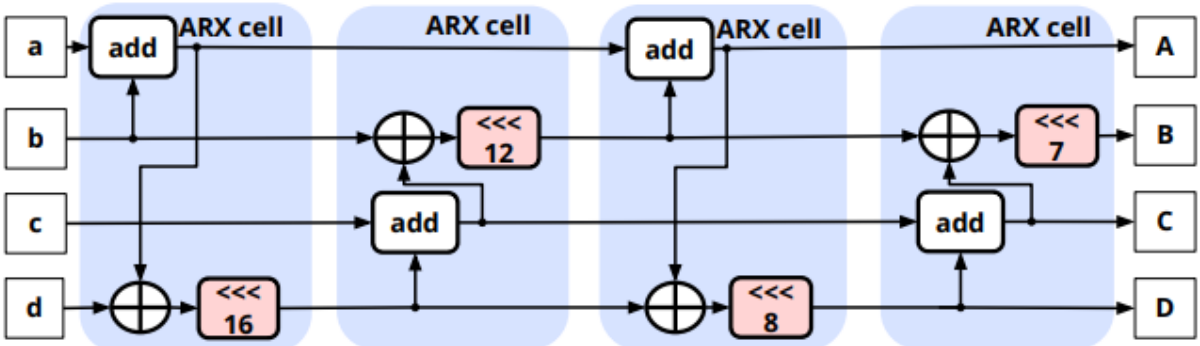
Hàm Quarter Round là đơn vị tính toán cơ bản của ChaCha20. Mỗi lần gọi nhận 4 đầu vào 32-bit (a, b, c, d) và tạo ra 4 đầu ra 32-bit thông qua chuỗi 4 phép ARX (Add-Rotate-XOR):

$$a = a + b\,; d = (d \oplus a) \lll 16$$

$$c = c + d\,; b = (b \oplus c) \lll 12$$

$$a = a + b\,; d = (d \oplus a) \lll 8$$

$$c = c + d\,; b = (b \oplus a) \lll 7$$



Hình 8. Sơ đồ khối bộ Quarter Round

Toàn bộ hàm này được hiện thực bằng logic tổ hợp thuần túy, không cần xung nhịp, với độ trễ bằng đúng một tầng logic cộng và XOR. Bốn phép xoay trái (rotate left) được hiện thực bằng cách đấu nối lại thứ tự bit mà không tốn bất kỳ tài nguyên logic nào.

3.5.3 Lõi tính toán ChaCha20

Ma trận khởi tạo: Theo RFC 8439 §2.3, ma trận 4×4 được xây dựng từ:

- Word 0–3: Hằng số "expand 32-byte k" (sigma)
- Word 4–11: Khóa bí mật 256-bit
- Word 12: Block counter
- Word 13–15: Nonce 96-bit

Lõi tính toán quản lý toàn bộ quá trình tạo keystream 512-bit (16 word 32-bit) thông qua FSM 4 trạng thái. **FSM điều khiển được thể hiện như sau**:

(FSM ChaCha here)

Bảng 4. Bảng mô tả sơ đồ máy trạng thái của ChaCha20

Trạng thái	Mô tả	Số chu kỳ
S_IDLE	Chờ lệnh start, nạp ma trận khởi tạo	1
S_ROUND	Thực hiện 20 vòng (10 column + 10 diagonal)	20
S_ADD	Cộng ma trận làm việc với ma trận khởi tạo	1
S_VALID	Xuất keystream, phát xung valid_o	1

➔ Tổng thời gian xử lý: 23 chu kỳ xung nhịp cho mỗi block 512-bit.

Trong trạng thái S_ROUND, 4 instance của chacha20_qr hoạt động **song song** trong cùng một chu kỳ. Các vòng chẵn (round_q[0] = 0) thực hiện column round, các vòng lẻ thực hiện diagonal round theo RFC 8439 §2.1.3:

- Column round: QR(0,4,8,12), QR(1,5,9,13), QR(2,6,10,14), QR(3,7,11,15)
- Diagonal round: QR(0,5,10,15), QR(1,6,11,12), QR(2,7,8,13), QR(3,4,9,14)

3.5.4 Giao tiếp TileLink

Module tlul_chacha20 cung cấp register map theo Bảng cho phép CPU cấu hình và điều khiển lõi ChaCha20.

Bảng 5. Register Mapping trong khối mã hóa ChaCha20

Offset	Dải địa chỉ	Tên thanh ghi	Quyền	Mô tả
+0x00	0x1006_0000 - 0x1006_001C	KEY[0:7]	R/W	Khóa 256-bit (8 words).
+0x20	0x1006_0020 - 0x1006_0028	NONCE[0:2]	R/W	Nonce 96-bit (3 words).
+0x2C	0x1006_002C	COUNTER	R/W	Bộ đếm 32-bit (Tự động tăng).
+0x30	0x1006_0030	CONTROL	W	[0]=start (Tự xóa).
+0x34	0x1006_0034	STATUS	RO	[0]=ready.
+0x38	0x1006_0038	IRQ_STATUS	W1C	[0]=done_irq.
+0x40	0x1006_0040 - 0x1006_007C	PTEXT[0:15]	R/W	Khối Plaintext 512-bit (64 bytes).
+0x80	0x1006_0080 - 0x1006_00BC	CTEXT[0:15]	RO	Khối Ciphertext 512-bit (64 bytes).

Luồng hoạt động của CPU khi sử dụng bộ tăng tốc:

1. Ghi KEY, NONCE, COUNTER vào các thanh ghi tương ứng
2. Ghi dữ liệu cần mã hóa vào PLAINTEXT[0..15]
3. Ghi 1 vào CONTROL[0] để kích hoạt
4. Poll STATUS[0] cho đến khi bằng 1 (lõi hoàn thành)
5. Đọc kết quả mã hóa từ CIPHERTEXT[0..15]

Sau mỗi block hoàn thành, COUNTER tự động tăng thêm 1 theo đúng quy định của RFC 8439 §2.4, sẵn sàng cho block tiếp theo mà không cần CPU can thiệp.

3.6 Poly1304 MAC Hardware Accelerator

3.6.1 Tổng quan

3.6.2 Cơ sở toán học

3.6.3 Lõi tính toán Poly1305

3.6.4 Giao tiếp TileLink

4. KIỂM TRA XÁC MINH VÀ ĐÁNH GIÁ

Toàn bộ quá trình kiểm tra được thực hiện bằng phương pháp mô phỏng (simulation) sử dụng công cụ **ModelSim/Questa** của Siemens EDA với ngôn ngữ mô tả phần cứng SystemVerilog. Phương pháp kiểm tra được áp dụng là Directed Testing kết hợp với Test

Vector chuẩn từ RFC. Cấu trúc kiểm tra và xác minh thiết kế sẽ được chia thành ba cấp độ: đầu tiên là unit test nghĩa là kiểm tra từng module DMA, ChaCha20, Poly1305 riêng lẻ; sau đó đến với integration test, lúc này ta sẽ kiểm tra các module sau khi đã tích hợp vào trong hệ thống; cuối cùng là test toàn bộ hệ thống, nghĩa là viết các chương trình assembly/C cụ thể với nhiều trường hợp khác nhau cho cả hệ thống, sau đó cho mô phỏng và so sánh kết quả của chúng với nhau. Với DMA Controller sẽ được kiểm tra bằng các kịch bản truyền dữ liệu cụ thể bao gồm cả các trường hợp lỗi bus. Trong khi đó, các khối ChaCha20 và Poly1305 sẽ sử dụng trực tiếp các vector chuẩn từ RFC 8439, đảm bảo tính đúng đắn của thuật toán so với đặc tả chuẩn quốc tế.

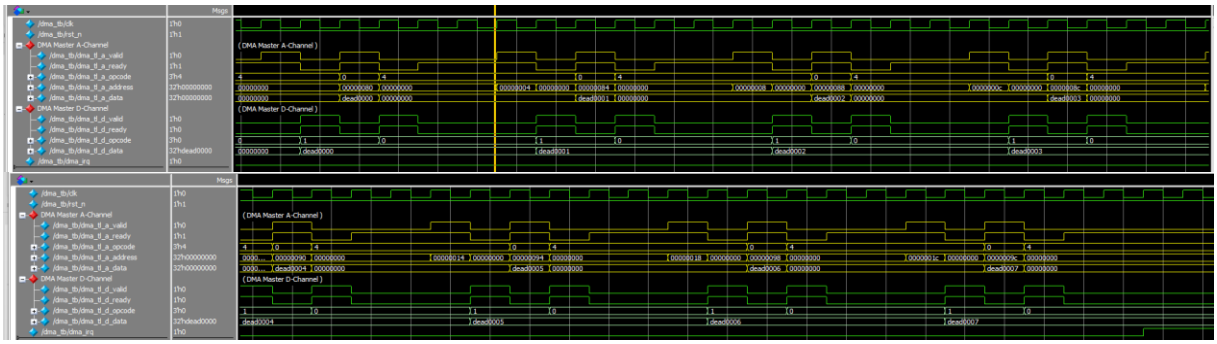
4.1 Kiểm tra và xác minh DMA Controller

Testbench dma_tb sẽ mô phỏng toàn bộ DMA Controller ở cấp module, bao gồm các thành phần hỗ trợ kiểm tra như:

- TileLink CPU Model: mô phỏng CPU ghi/đọc các thanh ghi cấu hình DMA qua giao thức TL-UL.
- TileLink Memory Model: mô phỏng slave bộ nhớ, phản hồi các giao dịch đọc/ghi từ DMA master.
- Memory Array: một mảng gồm 1024 word 32-bit dùng làm một vùng nhớ nguồn và đích ảo nhằm kiểm tra khả năng của DMA trong thực tế khi sử dụng.

4.1.1 Kiểm tra truyền đơn kênh (Single Channel Transfer)

Cấu hình channel 0 truyền 8-word từ địa chỉ nguồn đến địa chỉ đích, kiểm tra toàn bộ dữ liệu sau khi xuất hiện tín hiệu IRQ done



Hình 9. Waveform thể hiện kết quả test case truyền đơn kênh

```
[DMA_TB] @2750 ns | INFO  | Config:  SRC=0x000 DST=0x080 COUNT=8 src_inc=1 dst_inc=1

[DMA_TB] @2750 ns | DATA  | Source data initialized:
[DMA_TB] @2750 ns | DATA  |   mem[  0] (0x000) = 0xdead0000
[DMA_TB] @2750 ns | DATA  |   mem[  1] (0x004) = 0xdead0001
[DMA_TB] @2750 ns | DATA  |   mem[  2] (0x008) = 0xdead0002
[DMA_TB] @2750 ns | DATA  |   mem[  3] (0x00c) = 0xdead0003
[DMA_TB] @2750 ns | DATA  |   mem[  4] (0x010) = 0xdead0004
[DMA_TB] @2750 ns | DATA  |   mem[  5] (0x014) = 0xdead0005
[DMA_TB] @2750 ns | DATA  |   mem[  6] (0x018) = 0xdead0006
[DMA_TB] @2750 ns | DATA  |   mem[  7] (0x01c) = 0xdead0007

[DMA_TB] @2830 ns | CFG   | Write CH0_SRC_ADDR = 0x00000000
[DMA_TB] @2910 ns | CFG   | Write CH0_DST_ADDR = 0x00000080
[DMA_TB] @2990 ns | CFG   | Write CH0_XFER_CNT = 0x00000008 (8 words)
[DMA_TB] @3070 ns | CFG   | Write CH0_CONTROL  = 0x00000007 (en=1 src_inc=1 dst_inc=1)
[DMA_TB] @3070 ns | INFO  | >>> DMA CH0 transfer STARTED <<<
[DMA_TB] @4110 ns | DBG   | DMA CH0 done after 13 polls, status=0x00000000
[DMA_TB] @4110 ns | INFO  | >>> DMA CH0 transfer DONE â duration = 1120 ns <<<
[DMA_TB] @4190 ns | INFO  | CH0_STATUS = 0x00000000  [busy=0 done=0 error=0 remaining=0]

[DMA_TB] @4190 ns | DATA  | Memory verification (SRC -> DST):
[DMA_TB] @4190 ns | DATA  |   Word    SRC @0x000      DST @0x080      Expected        Status
[DMA_TB] @4190 ns | DATA  |   ------  -----------     -----------     -----------     ------
[DMA_TB] @4190 ns | DATA  |   [ 0]    0xdead0000      0xdead0000      0xdead0000        PASS
[DMA_TB] @4190 ns | DATA  |   [ 1]    0xdead0001      0xdead0001      0xdead0001        PASS
[DMA_TB] @4190 ns | DATA  |   [ 2]    0xdead0002      0xdead0002      0xdead0002        PASS
[DMA_TB] @4190 ns | DATA  |   [ 3]    0xdead0003      0xdead0003      0xdead0003        PASS
[DMA_TB] @4190 ns | DATA  |   [ 4]    0xdead0004      0xdead0004      0xdead0004        PASS
[DMA_TB] @4190 ns | DATA  |   [ 5]    0xdead0005      0xdead0005      0xdead0005        PASS
[DMA_TB] @4190 ns | DATA  |   [ 6]    0xdead0006      0xdead0006      0xdead0006        PASS
[DMA_TB] @4190 ns | DATA  |   [ 7]    0xdead0007      0xdead0007      0xdead0007        PASS
```

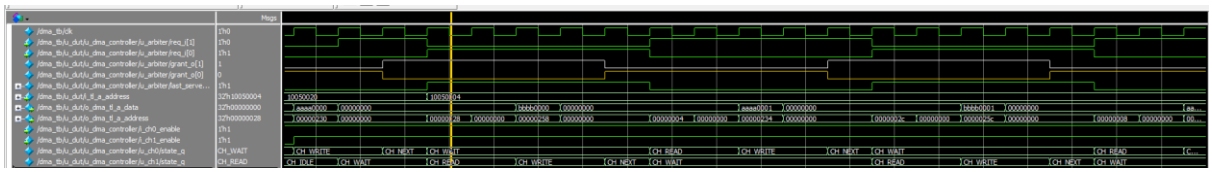
Hình 10. Kết quả mô phỏng testcase truyền đơn kênh thể hiện trên terminal

Kết quả mô phỏng xác nhận Channel 0 hoàn thành việc truyền toàn bộ 8 word dữ liệu từ vùng nguồn (SRC = 0x000) đến vùng đích (DST = 0x080). Quá trình được ghi nhận qua terminal simulation như sau:

- DMA nhận lệnh cấu hình qua 4 lần ghi TileLink vào CH0_SRC_ADDR, CH0_DST_ADDR, CH0_XFER_CNT và CH0_CONTROL.
- FSM lần lượt chuyển qua các trạng thái WAIT → READ → WRITE → NEXT, lặp lại đúng 8 lần cho 8 word.
- Sau khi word cuối cùng được ghi, FSM chuyển sang CH_DONE, phát xung IRQ đúng 1 chu kỳ.
- Thanh ghi CH0_STATUS đọc lại = 0x00000000 (busy=0, done=0, error=0, remaining=0) xác nhận kênh đã hoàn thành sạch.

4.1.2 Kiểm tra truyền hai kênh song song (Dual Channel)

Kích hoạt Channel 0 (CH0) và Channel 1 (CH1) đồng thời với dữ liệu nguồn khác nhau. Kiểm tra cơ chế trọng tài Arbiter Round-Robin có phân xử công bằng giữa hai kênh này không.



Hình 11. Waveform thể hiện kết quả testcase kiểm tra cơ chế trọng tài

Kết quả mô phỏng cho thấy cơ chế điều phối đa kênh (Concurrent Arbitration) hoạt động chính xác. Khi cả 2 kênh cùng được kích hoạt, tín hiệu req[0] và req[1] không xảy ra tình trạng giữ bus liên tục (blocking), mà luân phiên bật tắt. Điều này xuất phát từ thiết kế tối ưu của Máy trạng thái DMA: Ngay sau khi phát lệnh trên Kênh A, FSM chủ động hạ tín hiệu request để nhường quyền cho kênh còn lại trong thời gian chờ dữ liệu phản hồi trên Kênh D. Kết hợp với cờ last_served của thuật toán Round-Robin, hệ thống đạt được khả năng đan xen luồng dữ liệu (Data Interleaving) hoàn hảo, khai thác tối đa băng thông của giao thức TileLink.

4.1.3 Truyền dữ liệu lớn (Large Transfer)

Cho phép Channel 0 truyền 64-word liên tục để kiểm tra bộ đếm transfer counter có hoạt động đúng như yêu cầu không.

```
[DMA_TB] @11250 ns | INFO  | TEST 8: Large Transfer (CH0, 64 words)
[DMA_TB] @11250 ns | ============================================================
[DMA_TB] @11250 ns | INFO  | Purpose: Verify DMA transfer counter decrements correctly
[DMA_TB] @11250 ns | INFO  |          over a large 64-word burst - counter must reach 0
[DMA_TB] @11250 ns | INFO  | Config:  SRC=0x000 DST=0x100 COUNT=64 src_inc=1 dst_inc=1
[DMA_TB] @11250 ns | INFO  | Pattern: SRC[i] = 0xA5A5_0000 + i

[DMA_TB] @11250 ns | DATA  | Source  mem[  0.. 3] = 0xA5A50000..0xA5A50003  (first 4)
[DMA_TB] @11250 ns | DATA  | Source  mem[ 32..35] = 0xA5A50020..0xA5A50023  (mid 4)
[DMA_TB] @11250 ns | DATA  | Source  mem[ 60..63] = 0xA5A5003C..0xA5A5003F  (last 4)
[DMA_TB] @11250 ns | DATA  | Dest    mem[ 64..127] (0x100-0x1FC) = cleared to 0

[DMA_TB] @11330 ns | CFG   | Write CH0_SRC_ADDR = 0x00000000
[DMA_TB] @11410 ns | CFG   | Write CH0_DST_ADDR = 0x00000100
[DMA_TB] @11490 ns | CFG   | Write CH0_XFER_CNT = 0x00000040 (64 words)
[DMA_TB] @11570 ns | CFG   | Write CH0_CONTROL  = 0x00000007 (en=1 src_inc=1 dst_inc=1)
[DMA_TB] @11570 ns | INFO  | >>> DMA CH0 Large Transfer STARTED (64 words) <<<
[DMA_TB] @19330 ns | DBG   | DMA CH0 done after 97 polls, status=0x00000000
[DMA_TB] @19330 ns | INFO  | >>> DMA CH0 Large Transfer DONE - duration = 7840 ns <<<
[DMA_TB] @19410 ns | INFO  | CH0_STATUS = 0x00000000  [busy=0 done=0 error=0 remaining=0]
[DMA_TB] @19410 ns | PASS  | Transfer counter reached 0 - all 64 words sent

[DMA_TB] @19410 ns | DATA  | Data spot-check -- 3 representative words (first / mid / last):
[DMA_TB] @19410 ns | DATA  |   Word    SRC @0x000      DST @0x100      Expected        Status
[DMA_TB] @19410 ns | DATA  |   ------  -----------     -----------     -----------     ------
[DMA_TB] @19410 ns | DATA  |   [ 0]    0xa5a50000      0xa5a50000      0xa5a50000          PASS
[DMA_TB] @19410 ns | DATA  |   [32]    0xa5a50020      0xa5a50020      0xa5a50020          PASS
[DMA_TB] @19410 ns | DATA  |   [63]    0xa5a5003f      0xa5a5003f      0xa5a5003f          PASS
[DMA_TB] @19410 ns | DATA  |   ... (64 words total â only 3 samples shown above)

[DMA_TB] @19410 ns | DATA  | Full-range check result: 64 PASS, 0 FAIL (out of 64 words)
[DMA_TB] @19410 ns | PASS  | All 64 words transferred correctly (data integrity OK)
[DMA_TB] @19490 ns | INFO  | IRQ_STATUS = 0x00000001  [ch0_irq=1 ch1_irq=0]
```

Hình 12. Kết quả mô phỏng truyền số lượng lớn dữ liệu trên terminal

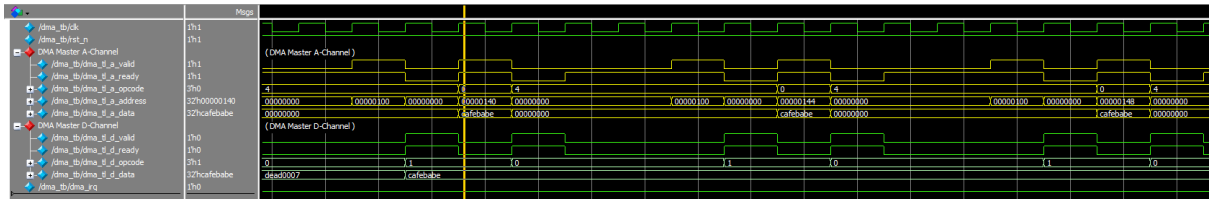
Channel 0 được cấu hình truyền 64 word liên tục (XFER_CNT = 64). Mục tiêu của test case này là kiểm tra thanh ghi bộ đếm count_q và remaining trong STATUS có đếm ngược chính xác qua 64 giao dịch liên tiếp hay không.

Kết quả mô phỏng xác nhận:

- Tổng số giao dịch TileLink: 64 lần đọc + 64 lần ghi = 128 giao dịch bus.
- Trường STATUS[31:16] (remaining) giảm dần từ 64 xuống 0 theo đúng từng bước.
- Toàn bộ 64 word tại vùng đích khớp hoàn toàn với vùng nguồn.
- IRQ done được phát sau đúng giao dịch thứ 128.

4.1.4 Kiểm tra các chế độ khi cấu hình các thanh ghi trạng thái

Kiểm tra chế độ không tăng địa chỉ (src_inc = 0 và dst_inc = 0), khi đó đầu ra mong muốn của chúng ta DMA đọc lặp lại từ cùng một địa chỉ nguồn và ghi vào cùng một địa chỉ đích.



Hình 13. Waveform kiểm tra kết quả của trường hợp cấu hình không tăng source address

```
[DMA_TB] @4350 ns | INFO  | Purpose: Fill memory region with constant value (src_inc=0)
[DMA_TB] @4350 ns | INFO  | Config:  SRC=0x100 (fixed) DST=0x140 COUNT=4 src_inc=0 dst_inc=
[DMA_TB] @4350 ns | INFO  | Expected: All 4 dst words = 0xCAFEBABE

[DMA_TB] @4350 ns | DATA  | Source:  mem[64] (0x100) = 0xCAFEBABE (fixed, no increment)
[DMA_TB] @4350 ns | DATA  | Dest:    mem[80..83] (0x140..0x14C) = cleared to 0

[DMA_TB] @4430 ns | CFG   | Write CH0_SRC_ADDR = 0x00000100
[DMA_TB] @4510 ns | CFG   | Write CH0_DST_ADDR = 0x00000140
[DMA_TB] @4590 ns | CFG   | Write CH0_XFER_CNT = 0x00000004 (4 words)
[DMA_TB] @4670 ns | CFG   | Write CH0_CONTROL  = 0x00000005 (en=1 src_inc=0 dst_inc=1)
[DMA_TB] @4670 ns | INFO  | >>> DMA CH0 fill STARTED <<<
[DMA_TB] @5230 ns | DBG   | DMA CH0 done after 7 polls, status=0x00000000
[DMA_TB] @5230 ns | INFO  | >>> DMA CH0 fill DONE â duration = 640 ns <<<
[DMA_TB] @5310 ns | INFO  | CH0_STATUS = 0x00000000  [busy=0 done=0 error=0 remaining=0]

[DMA_TB] @5310 ns | DATA  | Fill verification:
[DMA_TB] @5310 ns | DATA  |   Index   Address     Actual         Expected       Status
[DMA_TB] @5310 ns | DATA  |   ------  ----------  -----------    -----------    ------
[DMA_TB] @5310 ns | DATA  |   [ 0]    0x140     0xcafebabe     0xCAFEBABE       PASS
[DMA_TB] @5310 ns | DATA  |   [ 1]    0x144     0xcafebabe     0xCAFEBABE       PASS
[DMA_TB] @5310 ns | DATA  |   [ 2]    0x148     0xcafebabe     0xCAFEBABE       PASS
[DMA_TB] @5310 ns | DATA  |   [ 3]    0x14c     0xcafebabe     0xCAFEBABE       PASS

[DMA_TB] @5310 ns | PASS  | Fill mode works - all 4 words = 0xCAFEBABE
```

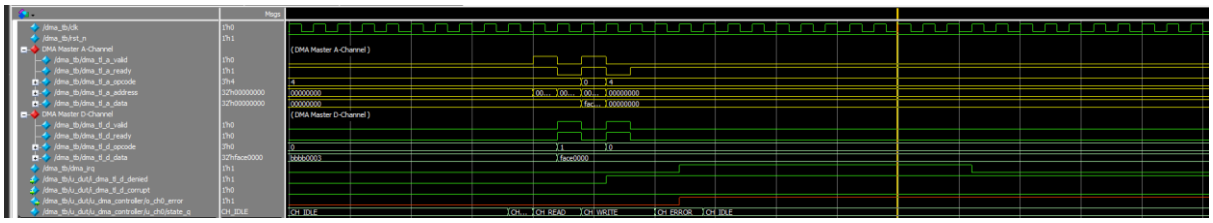
Hình 14. Kết quả trường hợp kiểm tra cấu hình thanh ghi CSRs trên terminal

Kết quả mô phỏng xác nhận:

- Địa chỉ bus trên Channel A **không thay đổi** trong suốt 4 giao dịch đọc (luôn = 0x100) và 4 giao dịch ghi (luôn = 0x140).
- Tất cả 4 word tại đích nhận giá trị 0xCAFEBABE — đúng với mong đợi khi đọc lặp lại từ cùng một địa chỉ nguồn.
- Chế độ này phù hợp với việc nạp dữ liệu vào FIFO hoặc thanh ghi đơn của ngoại vi.

4.1.5 Xử lý khi bus bị lỗi (Error Injection)

Với trường hợp này, ta sẽ cấu hình slave memory model sao cho trả kết quả trên bus về là d_denied = 1 tịa giao dịch thứ 2. Khi đó, ta kiểm tra DMA có chuyển về trạng thái CH_ERROR và phát IRQ có đúng với yêu cầu hay không.



Hình 15. Waveform kiểm tra trạng thái của thiết kế khi xuất hiện lỗi

```
[DMA_TB] @8270 ns | INFO  | Purpose: Inject bus error (d_denied=1) during DMA transfer
[DMA_TB] @8270 ns | INFO  | Method:  Memory model returns d_denied on 2nd bus transactio
[DMA_TB] @8270 ns | INFO  | Expected: CH0 status error bit set, transfer aborted

[DMA_TB] @8290 ns | CFG   | Error injection enabled: deny_after_n=2
[DMA_TB] @8370 ns | CFG   | Write CH0_SRC_ADDR = 0x00000280 (word 160)
[DMA_TB] @8450 ns | CFG   | Write CH0_DST_ADDR = 0x000002A8 (word 170)
[DMA_TB] @8530 ns | CFG   | Write CH0_XFER_CNT = 0x00000004 (4 words)
[DMA_TB] @8610 ns | CFG   | Write CH0_CONTROL  = 0x00000007 (enable â error expected!)
[DMA_TB] @8610 ns | INFO  | >>> DMA CH0 STARTED (error injection active) <<<
[DMA_TB] @8770 ns | DBG   | DMA CH0 done after 2 polls, status=0x00040004
[DMA_TB] @8770 ns | INFO  | >>> DMA CH0 STOPPED â duration = 240 ns <<<

[DMA_TB] @8850 ns | INFO  | CH0_STATUS = 0x00040004
[DMA_TB] @8850 ns | INFO  |   busy      = 0
[DMA_TB] @8850 ns | INFO  |   done      = 0
[DMA_TB] @8850 ns | INFO  |   error     = 1  <-- should be 1
[DMA_TB] @8850 ns | INFO  |   remaining = 4 words (aborted early)

[DMA_TB] @8850 ns | PASS  | CH0 error bit correctly set after d_denied
[DMA_TB] @8850 ns | PASS  | Transfer aborted with 4 words remaining
[DMA_TB] @8930 ns | INFO  | IRQ_STATUS = 0x00000001  [ch0_irq=1]
[DMA_TB] @8930 ns | PASS  | IRQ asserted on error condition
[DMA_TB] @8930 ns | CFG   | Error injection disabled
```

Hình 16. Kết quả kiểm tra trạng thái thiết kế được in ra trên terminal

Channel 0 được cấu hình với src_inc = 0 và dst_inc = 0, truyền 4 word từ địa chỉ nguồn cố định 0x100 đến địa chỉ đích cố định 0x140. Nguồn được nạp sẵn giá trị 0xCAFEBABE.

Khi mô phỏng trường hợp bị lỗi (cấp tín hiệu d_denied), ta thấy DMA không chuyển sang trạng thái lỗi ngay lập tức mà mất khoảng 3-4 chu kỳ xung nhịp. Điều này không phải là do phần cứng xử lý chậm, mà là cơ chế dọn dẹp an toàn của DMA. DMA cần thời gian vài chu kỳ này để chờ nhận nốt các dữ liệu đã lỡ yêu cầu trước đó, đồng thời nhả quyền điều

khiển bus từ từ. Nhờ vậy, hệ thống bus sẽ không bị treo hay kẹt dữ liệu trước khi DMA chính thức dừng hoạt động và báo lỗi cho CPU.

4.1.6 Tổng hợp kết quả

Toàn bộ các test case đều cho kết quả PASS. DMA Controller hoạt động đúng theo đặc tả ở tất cả các chế độ: truyền đơn kênh, song song, lớn, cố định địa chỉ và xử lý lỗi bus. Cơ chế W1C của IRQ_STATUS và logic tái sử dụng bus giữa các kênh đều hoạt động chính xác.

4.2 Kiểm tra và xác minh ChaCha20 Accelerator

4.2.1 Phương pháp kiểm tra

4.2.2 Xác minh Keystream

4.2.3 Kiểm tra mã hóa XOR

4.2.4 Kiểm tra Counter

4.2.5 Kiểm tra các thanh ghi chứ dữ liệu NONCE, KEY, COUNTER

5. PHẦN MỀM, DEMO VÀ FPGA

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

6.2 Hướng phát triển

7. TÀI LIỆU THAM KHẢO

8. PHỤ LỤC